

Adding React to Flask (CodeGrade)

 <https://github.com/learn-co-curriculum/python-p4-adding-react-to-flask>  <https://github.com/learn-co-curriculum/python-p4-adding-react-to-flask/issues/new>

Learning Goals

- Use React and Flask together to build beautiful and powerful web applications.
- Organize client and server code so that it is easy to understand and maintain.

Key Vocab

- **Full-Stack Development:** development of a frontend and a backend for an application. True full-stack development includes a database, a logic/server layer, and a frontend built in JavaScript, HTML, and CSS.
- **Backend:** the layer of a full-stack application that handles business logic and other programmatic tasks that users do not or should not see. Can be written in many languages, including Python, Java, Ruby, PHP, and more.
- **Frontend:** the layer of a full-stack application that users see and interact with. It is always written in the frontend languages: JavaScript, HTML, and CSS. (*There are others now, but they are based on these three.*)
- **Cross-Origin Resource Sharing (CORS):** a method for a server to indicate any ports (or other identifiers) for servers that can share its resources.
- **Transmission Control Protocol (TCP):** a protocol that defines how computers send data to each other. A connection is formed and stays active until the applications on either end have finished sending data to one another.
- **Hypertext Transfer Protocol (HTTP):** a stateless protocol where applications communicate for the length of time that it takes for data to be transferred.
- **Websocket:** a protocol that allows clients and servers to communicate with one another in both directions. The bidirectional nature of websocket communication allows a connected state to be generated and the connection maintained until it is terminated by one side. This

allows for speedy and seamless connections between frontends and backends.

Introduction

Earlier in this phase, we used React and Flask together for two different applications: Chatterbox, a messenger with CRUD functionality, and Plantsy, a plant shop with CRUD functionality through a RESTful backend. In these labs, we focused on the server-side (Python) code. Now, let's take a closer look at the JavaScript that creates the user interface.

Setup

This lesson contains the solution code from the Chatterbox lab. To run the application, open two terminal windows. In the first, enter the `server/` directory and run:

- `pipenv install && pipenv shell` to enter your virtual environment.
- `export FLASK_APP=app.py` and `export FLASK_RUN_PORT=5555` to configure your Flask environment.
- `flask db upgrade` to generate your database.
- `python seed.py` to populate it.
- `python app.py` to run your development server.

In the second window, enter the `client/` directory and run:

- `npm install` to retrieve the React project's dependencies.
- `npm start` to start your development server and open the application.

NOTE: There's a lot more to keep track of now! When you have to run several commands to start working, it's useful to write scripts to automate the startup process. Refer back to "Configuring Python Applications" in Phase 3 if you need help getting started! Don't worry about messing things up- you can always re-fork the lesson if you need to.

In your browser, you should see the Chatterbox app in all its glory:

Chatterbox



Search...

Stephanie 4:08:20 PM

Cause music than eye professor create green.

Stephanie 4:08:20 PM

But professor of

Once you've confirmed that the application is running correctly, open up the `client/` directory to explore our JavaScript code.

The Client-Server Model

In previous lessons, we've discussed the functions of the client and server in web applications: the client handles what goes on in the browser (i.e. the tasks controlled by the user) and the server handles data from the database and hidden tasks that the user doesn't need to or shouldn't see. We use this separation of tasks to inform how we structure our applications in development.

Within our base directory where we initialize Git, we create our basic documentation files like `README.md` and two directories: `client/` and `server/`. You may prefer to name them more descriptively, like `chatterbox-client/` and `chatterbox-server/`. (Some people find this redundant, others informative. Ultimately up to you!)

Inside of the `client/` directory, you will use Node to create the skeleton for your client-side code and install dependencies. In the `server/` directory, you will use Pipenv to install dependencies, then Flask to create your server-side application and database. We will explore this in more detail in the next lesson.

When development servers are run for both sides, the two can communicate over Transmission Control Protocols (TCP) such as HTTP and Websocket. Any TCP connection stays active until the two sides are done sending data to one another; in HTTP, the connection ends every time a message is sent. This is what we see in using `fetch()`. Websocket, another protocol, keeps the connection open until it is explicitly ended by either side- we will learn more about this with `socket.io` later in this module.

React `fetch()`

React uses a function called `fetch()` to retrieve data from APIs at other URLs. In order to get data with fetch, we put the command inside of a `useEffect` hook as seen in `client/src/components/App.js`. We include an empty array as a second argument (dependencies) to tell

`useEffect` to only run `fetch()`, an asynchronous operation, on the first render of `App`.

```
// client/src/components/App.js

function App() {
  ...
  useEffect(() => {
    fetch("http://127.0.0.1:5555/messages")
      .then((r) => r.json())
      .then((messages) => setMessages(messages));
  }, []);
  ...
}
```

As we can see, this has the React application looking for a resource at `http://127.0.0.1:5555/messages`: our Flask API. The response data is `then()` converted to JSON if it is not already in that format, `then()` that data is used to populate the application with messages.

We can throw much more into our chain here- for instance, if we're looking for a "200: OK" response from the server:

```
// client/src/components/App.js

function App() {
  ...
  useEffect(() => {
    fetch("http://127.0.0.1:5555/messages")
      .then(r => {
        if (r.ok) {
          return r.json()
        }
        throw r;
      })
      .then((messages) => setMessages(messages))
  }, []);
  ...
}
```

```
}, []);
...
```

It is wise when developing full-stack applications to look for the correct data formats and response codes from the server as you move forward. Since you wrote the backend yourself, this is *much* easier than doing so while `fetch()` ing data from someone else's API and will help your users understand how to avoid errors in the future. In the example above, we simply `throw r` because it receives any error message we catch and send forward from our Flask backend!

POST , PATCH , DELETE with `fetch()`

`fetch()` defaults to using `GET` as its HTTP method. If we want to carry out functions other than basic retrieves, we need to specify that and our message format

```
//client/src/components/NewMessage.js
```

```
function NewMessage({ currentUser, onAddMessage }) {
  const [body, setBody] = useState("");

  function handleSubmit(e) {
    e.preventDefault();

    fetch("http://127.0.0.1:5555/messages", {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
      },
      body: JSON.stringify({
        username: currentUser.username,
        body: body,
      }),
    })
      .then((r) => r.json())
```

```

    .then((newMessage) => {
      onAddMessage(newMessage);
      setBody("");
    });
  }
  ...

```

Rather than `useEffect`, here we have a hook called `onAddMessage` passed to the `NewMessage` element that updates the entire app when a message is confirmed to have been created. `handleSubmit()` is invoked each time a user hits the "Send" button and generates a `POST` request with the body of the `NewMessage` element.

`fetch()` takes an optional object as a second argument after the Flask API's URL with an HTTP request method, headers to specify the format of the message, and a message body. (`PATCH` requests need these elements as well!)

After generating a new record in the database, Flask returns a response that is assigned to `r` and converted to JSON if necessary. Finally, we invoke `onAddMessage` to update the app with this new message and reset the form to be empty and ready for new input.

CORS Recap

Cross-Origin Resource Sharing, or CORS, is a mechanism that lets a server (the Flask application in our case) specify URL patterns other than its own from which the client should be allowed to load resources. This is carried out in HTTP headers, but we typically handle it in a more automated fashion with extensions like Flask-CORS.

The Fetch API follows the same-origin policy, which enforces that resources can only be loaded from URL patterns owned by the application sending the request. This is why we need to use CORS in our Flask application, as seen in `server/app.py`:

```

# server/app.py

from flask import Flask
from flask_cors import CORS
...
app = Flask(__name__)

```

...
CORS(app)

Conclusion

This has been a brief review of concepts from Phase 2 and the beginning of Phase 4. Hopefully, this served as a good reminder of how to use `fetch()`, process `fetch()` data, and implement CORS! We will elaborate on these concepts and learn how to improve connections between full-stack application clients and servers in the coming lessons.

Resources

- [Client-Server Model - GeeksforGeeks](https://www.geeksforgeeks.org/client-server-model/)  (<https://www.geeksforgeeks.org/client-server-model/>)
- [Using the Fetch API - Mozilla](https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch)  (https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch)
- [Cross-Origin Reference Sharing \(CORS\) - Mozilla](https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS)  (<https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS>)

This tool needs to be loaded in a new browser window

Load Adding React to Flask (CodeGrade) in a new window